

Module 4 — Spring MVC

A introduction to the Spring MVC web layer — connecting everything learned in Spring Core and Spring JDBC to a clean, structured web application architecture.

SPRING FRAMEWORK SERIES

MODULE 4 OF 4

Course Structure & Scope

This module covers **12-15 slides** designed for a 1-hour session (~4 minutes per slide), leaving time for questions and a live demo. Since freshers have already completed **Spring Core** and **Spring JDBC**, this module focuses on how the web layer connects everything learned so far.

Spring Core ✓

Dependency Injection, Beans,
ApplicationContext

Spring JDBC ✓

JdbcTemplate, DAO pattern, Database
access

Spring MVC ← Now

Web layer, Controllers, Views, Request flow

Spring MVC Overview

Spring MVC is a web framework built on the **Model-View-Controller** pattern. It provides a structured way to handle HTTP requests, process business logic, and render responses — all within the Spring ecosystem.



Model

Carries data between Controller and View



View

Renders the response (JSP, Thymeleaf, HTML)



Controller

Handles requests and coordinates logic

Why Spring MVC?

Traditional Java Web App

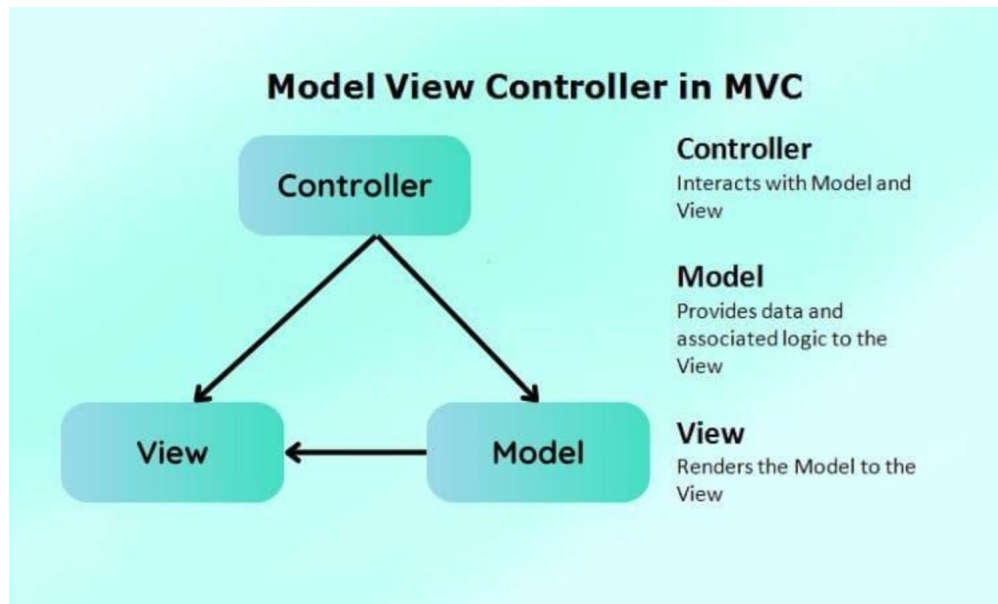
Browser → Servlet → JDBC → Database

- Large, bloated Servlet classes
- Business logic inside Servlets
- Difficult to maintain
- Tight coupling throughout

Spring MVC Solution

Clean separation of responsibilities:

- Controller — handles web requests
- Service — processes business logic
- Repository — manages data access
- Separation of concerns throughout



MVC Architecture

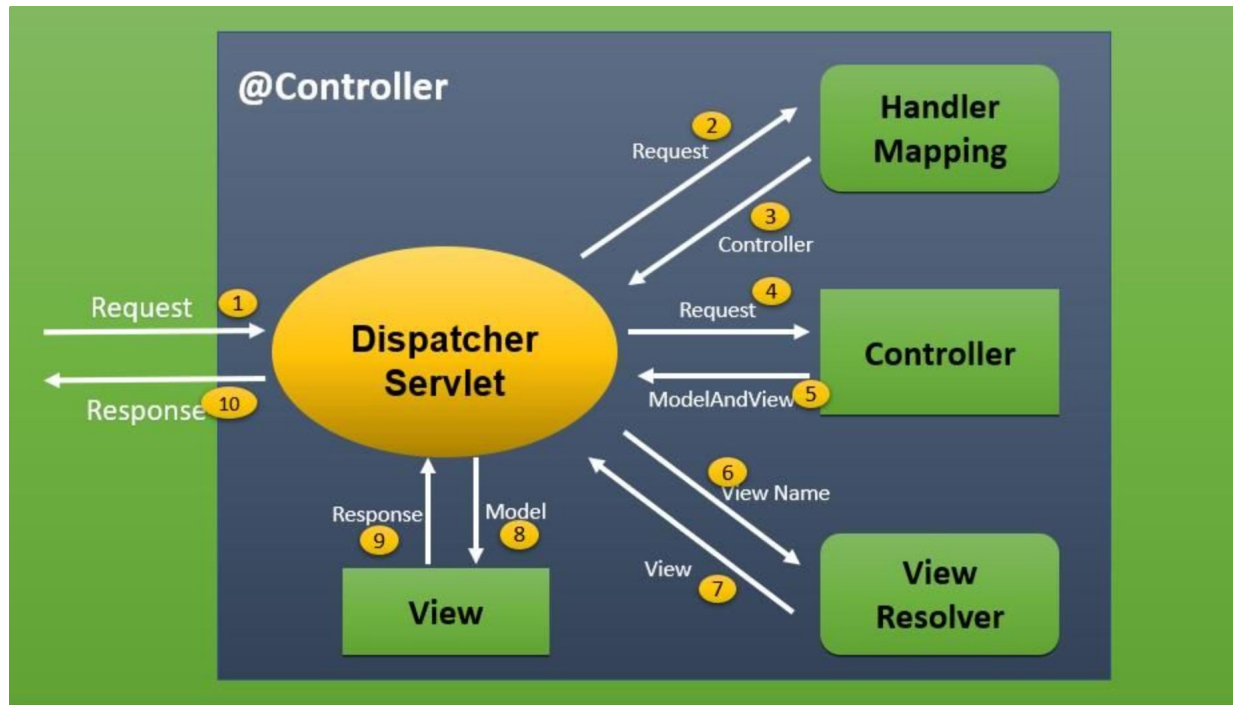
The three components work together in a clear triangle of responsibility. Using an **Online Student Portal** as an example: the Controller receives the request → Service processes data → Repository reads the database → View displays the result.



Spring MVC Components

Each component has a distinct responsibility in the framework. Together they form a clean, layered architecture.

1	DispatcherServlet Front controller — receives all incoming requests
2	Controller Handles specific URL mappings and request logic
3	Service Contains business logic, called by the Controller
4	Repository Data access layer — interacts with the database
5	View Resolver Maps logical view names to actual JSP/template files

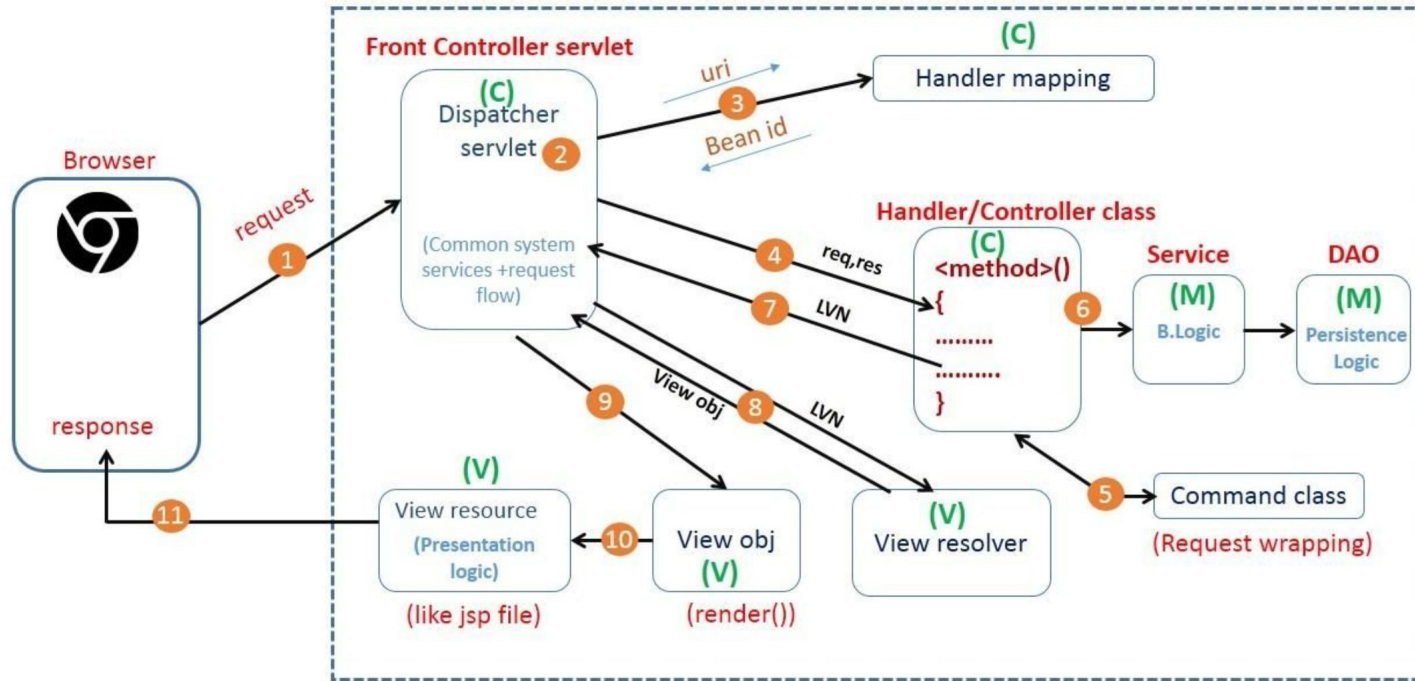


The DispatcherServlet — Front Controller

Every request first reaches the **DispatcherServlet**. It implements the **Front Controller Pattern** — a single entry point that delegates to the appropriate controller and manages the full request lifecycle.

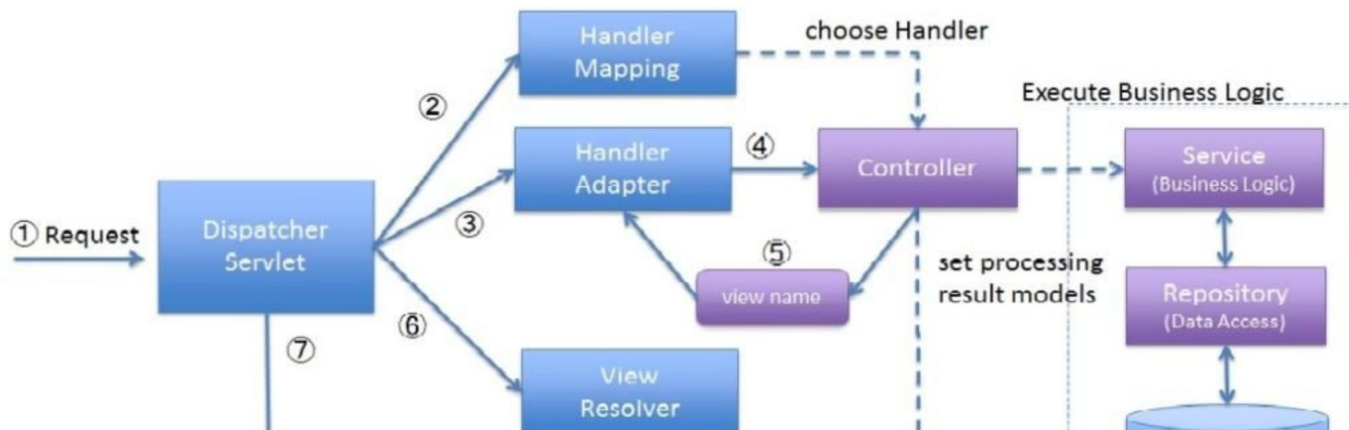
- 1 Receive Request
- 2 Find Controller
- 3 Invoke Method
- 4 Get Response
- 5 Return View

Spring MVC web application (java web application)



Complete Request Flow

When a student clicks <http://localhost:8080/students>, the full flow is: **Browser** → **DispatcherServlet** → **StudentController** → **StudentService** → **StudentRepository** → **Database** → **Model** → **JSP** → **Browser**. Each layer has a single, clear responsibility.



Creating Controllers

A Spring MVC controller is a plain Java class annotated with `@Controller`. The `@RequestMapping` annotation maps a URL path to the class or method.

```
@Controller
@RequestMapping("/student")
public class StudentController {
    // handler methods go here
}
```

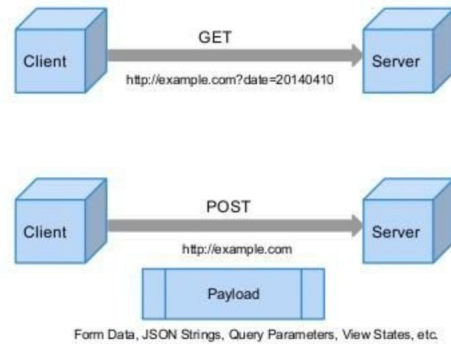
@Controller

Registers the class as a Spring MVC controller — Spring detects it via component scanning.

@RequestMapping

Maps the base URL path `/student` to this controller for all incoming requests.

GET vs. POST



Handling Requests — GET vs POST

```
@GetMapping("/list")
```

GET — Retrieve data from the server. Used to display a list of students or load a form page. Data is passed via the URL.

```
@PostMapping("/save")
```

POST — Submit data to the server. Used to save a new student record. Data is sent in the request body (payload), not the URL.

Passing Data — @PathVariable vs @RequestParam

Two common ways to pass data to a controller method from the URL:

Feature	@PathVariable	@RequestParam
URL Style	/student/101	/student?id=101
Location	URL path segment	Query parameter
Required?	Mandatory	Optional
Best for	Resource identity	Filters / search

Model & View — Data Flow to the Browser

The Model

Transfers data from the Controller to the View. The controller adds attributes that the JSP can read and render as HTML.

```
model.addAttribute("students", students);
```

The View Resolver

Maps a logical view name returned by the controller to an actual file on disk:

```
"studentList" →  
/WEB-INF/views/studentList.jsp
```

Common view technologies: **JSP**, **Thymeleaf**, plain **HTML**.



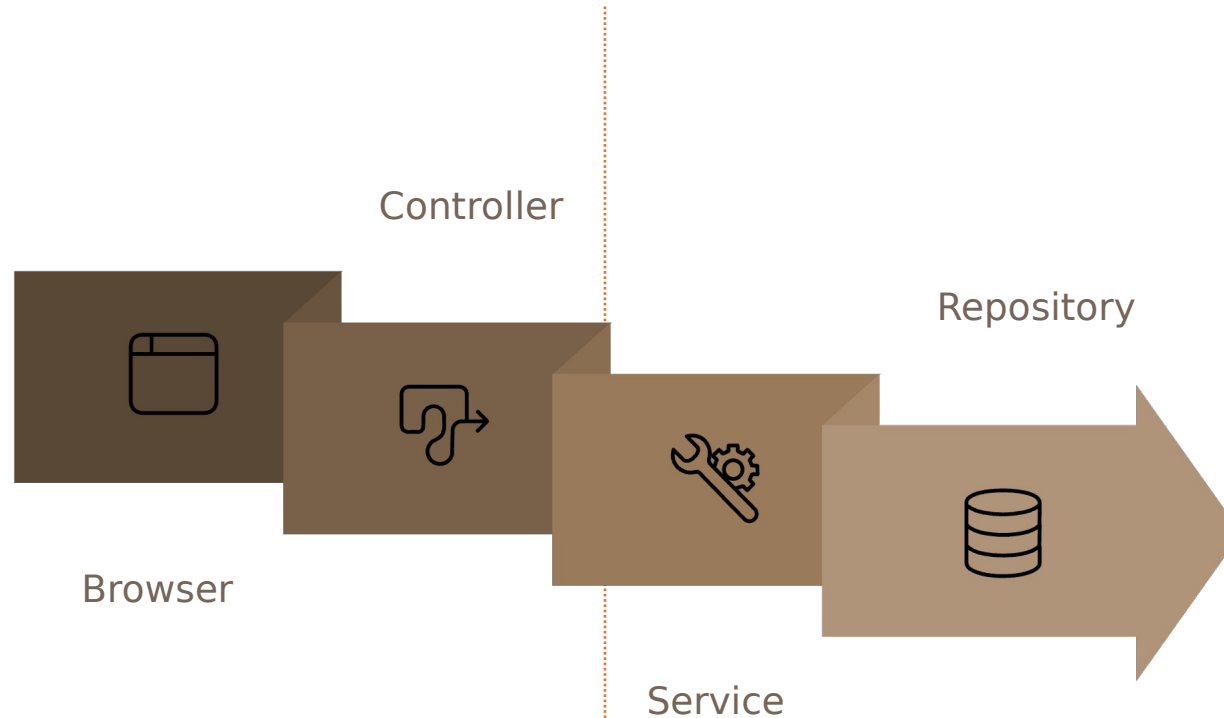
Spring MVC vs REST API

Spring MVC returns **HTML pages** for browser consumption. REST APIs return **JSON/XML** for any client (browser, mobile, other services). REST APIs will be covered in the next module.

Feature	Spring MVC	REST API
Response Format	HTML	JSON / XML
Client	Browser	Browser / Mobile / Any
Uses JSP?	Yes	No
Purpose	Web UI	Data API

Mini Application — Complete Layered Architecture

This diagram shows how all three completed modules work together in a single, cohesive Spring application — from browser request to database and back.



Spring Core

Dependency Injection & Bean management

Spring JDBC

JdbcTemplate & Repository layer

Spring MVC

Web layer, Controllers & Views

Module Summary

After completing this module, students should confidently understand the following concepts:

MVC Pattern & DispatcherServlet

The architectural pattern and the front controller that routes every request

Request Flow & Annotations

End-to-end flow plus key annotations: @Controller, @RequestMapping, @GetMapping, @PostMapping, @PathVariable, @RequestParam

Controller, Model & View

How each component is defined, annotated, and wired together

MVC vs REST

Key differences — preparing students for the upcoming REST API module

Recommended Visuals for This Module

These six diagrams significantly improve student comprehension when used alongside the slides:

01

MVC Architecture Diagram

Introduces the Model–View–Controller triangle

03

Layered Architecture Diagram

Browser → Controller → Service → Repository → Database

05

Controller–Model–View Relationship

Explains how data flows to the view

02

DispatcherServlet Request Flow

Shows how every request is routed step by step

04

GET vs POST Illustration

Differentiates request methods visually

06

MVC vs REST Comparison

Prepares students for the upcoming REST module



This 14-slide set fits comfortably into a 1-hour session — approximately 4 minutes per slide, with time for questions and a short live demonstration.